

How Fetch API works (151)

```
fetch(URL, options);
```

the `fetch()` function returns a promise so you need to add the `.then()` and `.catch()` methods.

When the request returns a response, the `then()` method will be called.

If the request returns an error, then the `catch()` method will be executed.

Inside the `.then()` and `.catch()` methods, you pass a callback function to execute when the respective methods are called.

```
fetch('https://jsonplaceholder.typicode.com/todos/1')
  .then(response => console.log(response))
  .catch(error => console.log(error));
```

Here, you can see that the `body` property contains a `ReadableStream`. To use the `ReadableStream` in our JavaScript application, we need to convert it to JSON format first, so let's do it:

```
fetch('https://jsonplaceholder.typicode.com/todos/1')
  .then(response => response.json())
  .then(data => console.log(data))
```

The `json()` method converts the `ReadableStream` into a JavaScript object. The `data` variable above will be printed as follows:

```
{
  userId: 1,
  id: 1,
  title: "delectus aut autem",
  completed: false
}
```

When you send a POST method, you need to set the request header and body properties to ensure a smooth process.

For the header, you need to add the `Content-Type` property and set it to `application/json`.

The data you want to send should be put inside the `body` property in a JSON format. See the example below:

```
fetch('https://jsonplaceholder.typicode.com/todos', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ title: 'Learn JavaScript' }),
})
  .then(response => response.json())
  .then(data => console.log(data))
```

In the example above, we send a POST request to create a new todo task. The title of the task is `Learn JavaScript`. The response from the `typicode` API would be similar as follows:

```
{
  title: 'Learn JavaScript',
  id: 201
}
```